



طراحی سیستم‌های دیجیتال

بهار ۱۴۰۵

استاد: دکتر اجلالی

محسن صلاح - ۴۰۳۱۰۶۲۳۸

تمرین شماره ۴

مالتی پلکسر حافظه‌دار

۱ هدف تمرین

در این تمرین، با تست کردن کد مالتی پلکسر ۲ به ۱ که در کلاس نوشته شد، به بررسی یکی از قواعد کد نویسی قابل سنتز در توصیف رفتاری می‌پردازیم. این قاعده می‌گوید: «هنگام توصیف رفتاری یک مدار ترکیبی بعد از @ (تنها یک @ داریم و آن هم پس از always هست)، باید تمام ورودی‌های مدار ترکیبی را در sensitivity list قرار دهیم.»

۲ کدهای Verilog

۱.۲ ماژول مالتی پلکسر (MUX_2_1.v)

در کد این مالتی پلکسر (که توصیف رفتاری است) عمداً ورودی sel (پایه‌ی انتخاب) از sensitivity list حذف شده. بنابراین مدار فقط به تغییرات دو ورودی I0 و I1 واکنش نشان می‌دهد و با صرفاً تغییر دادن sel، خروجی مدار تغییر نمی‌کند. کد این ماژول به این صورت می‌باشد:

```
1 module MUX_2_1 (I0, I1, sel, O);
2
3     input [7:0] I0, I1;
4     input sel;
5     output [7:0] O;
6     reg [7:0] O;
7
8     always @(I0, I1)
9         if (sel == 1)
10             O = I1;
11         else
12             O = I0;
13
14 endmodule
```

بر اساس تعریف، «اگر یک مدار به ازای ورودی‌های کاملاً یکسان در زمان‌های گوناگون خروجی‌های متفاوت داشته باشد، می‌گوییم حافظه دارد.» در این مثال هم چون ورودی sel از sensitivity list حذف شده، به صورت ناخواسته مدار حافظه‌دار شده که می‌توانیم با یک تست‌بنچ این موضوع را نشان دهیم.

۲.۲ تست‌بنچ (testbench.v)

می‌خواهیم در این تست‌بنچ حالتی ایجاد کنیم که به ازای ورودی‌های کاملاً یکسان در زمان‌های متفاوت، ماژول مالتی‌پلکسر دو خروجی متفاوت بدهد. (و نتیجه بگیریم که این ماژول حافظه دارد.) برای این کار می‌توانیم از این استراتژی استفاده کنیم: ابتدا مقادیر اولیه را به MUX می‌دهیم ($sel = 1$)، سپس sel را تغییر می‌دهیم تا خروجی‌های بعدی از $I0$ بیایند. برای اینکه مقدار خروجی تغییر کند، باید $I0$ تغییر کند؛ پس $I0$ را به یک مقدار ثانویه تغییر داده و سپس به حالت اولیه برمی‌گردانیم تا خروجی آپدیت شود. در نهایت هم $sel = 1$ می‌کنیم تا ورودی‌ها کاملاً مشابه حالت اولیه شوند.

```
1 `timescale 1ns / 1ns
2
3 module testbench;
4     reg [7:0] I0;
5     reg [7:0] I1;
6     reg sel;
7     wire [7:0] O;
8
9     MUX_2_1 mux (I0, I1, sel, O);
10
11     initial begin
12         $dumpfile("waves.vcd");
13         $dumpvars(0, testbench);
14
15         $display("Time\tI0\tI1\t sel\tO");
16         $display("-----");
17     end
18
19     always @(I0, I1, sel, O) begin
20         #0.1;
21         $display("%4dt\tI0\tI1\t sel\tO", $time, I0, I1, sel, O);
22     end
23
24     initial begin
25         I0 = 8'hAA; I1 = 8'hBB; sel = 1;
26         #20;
27
28         sel = 0;
29         #20;
30
31         I0 = 8'h00;
32         #20;
33
34         I0 = 8'hAA;
35         #20;
36
37         sel = 1;
38         #20;
39
40         $finish;
41     end
42 endmodule
```

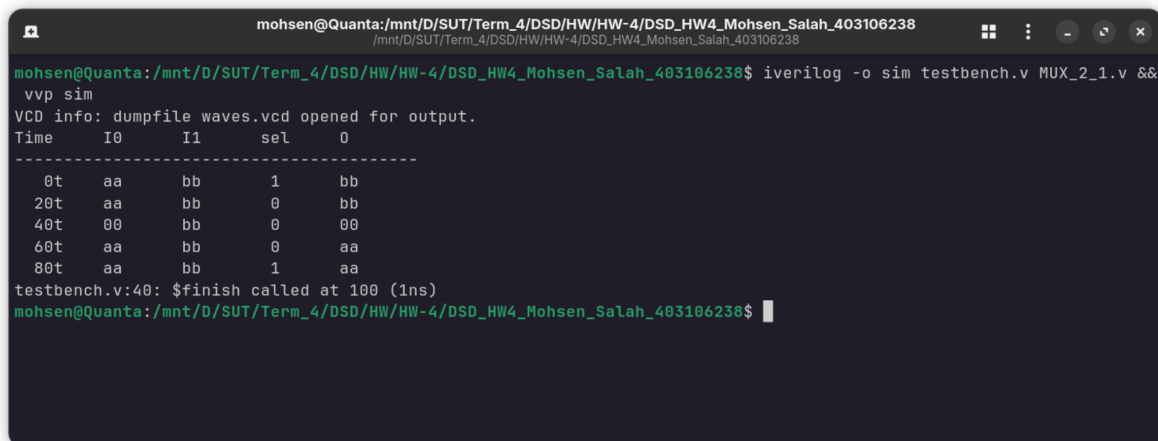
۳.۲ اصلاح مالتی پلکسر (MUX_2_1_FIXED.v)

برای اصلاح ماژول مالتی پلکسر، کافی است ورودی sel را به sensitivity list اضافه کنیم. بنابراین کد اصلاح شده به این صورت خواهد بود:

```
1 module MUX_2_1 (I0, I1, sel, O);
2
3     input [7:0] I0, I1;
4     input sel;
5     output [7:0] O;
6     reg [7:0] O;
7
8     always @(I0, I1) // CHANGED
9         if (sel == 1)
10             O = I1;
11         else
12             O = I0;
13
14 endmodule
```

۳ اجرای کد

کدهای نوشته شده را کامپایل و اجرا می‌کنیم. خروجی تست‌بنچ به این صورت می‌باشد:



```
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-4/DSD_HW4_Mohsen_Salah_403106238$ iverilog -o sim testbench.v MUX_2_1.v &&
vvp sim
VCD info: dumpfile waves.vcd opened for output.
Time      I0      I1      sel      O
-----
0t        aa      bb      1        bb
20t        aa      bb      0        bb
40t        00      bb      0        00
60t        aa      bb      0        aa
80t        aa      bb      1        aa
testbench.v:40: $finish called at 100 (1ns)
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-4/DSD_HW4_Mohsen_Salah_403106238$
```

شکل ۱: خروجی تست‌بنچ نوشته شده برای کد اصلی

همان طور که مشاهده می‌شود، در زمان‌های ۲۰ و ۸۰، ورودی مالتی پلکسر کاملاً یکسان است ($I0 = 0xAA$ ، ولی در زمان ۲۰، $I1 = 0xBB$ و $sel = 1$) و در زمان ۸۰، $O = 0xAA$. پس خروجی مدار با ورودی‌های کاملاً یکسان در دو زمان مختلف متفاوت است و نتیجه می‌گیریم که مدار حافظه دارد.

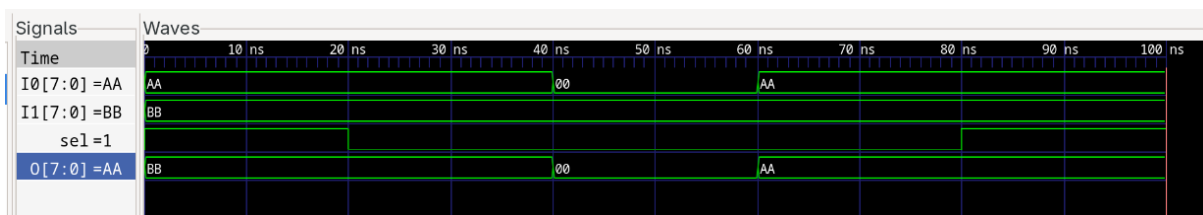
خروجی تست‌بنچ برای نسخه‌ی اصلاح شده نیز به این صورت می‌باشد:

```
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-4/DSD_HW4_Mohsen_Salah_403106238$ iverilog -o sim testbench.v MUX_2_1_FIXED.v &&
vvp sim
VCD info: dumpfile waves.vcd opened for output.
Time      I0      I1      sel      0
-----
0t        aa      bb      1        bb
20t        aa      bb      0        aa
40t        00      bb      0        00
60t        aa      bb      0        aa
80t        aa      bb      1        bb
testbench.v:40: $finish called at 100 (1ns)
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-4/DSD_HW4_Mohsen_Salah_403106238$
```

شکل ۲: خروجی تست‌بنچ نوشته شده برای کد اصلاح شده

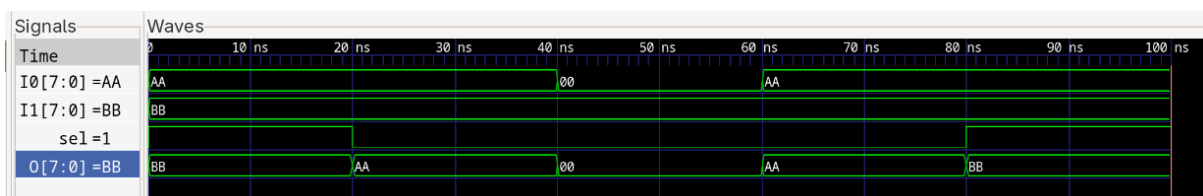
که می‌بینیم در این نسخه کد به درستی عمل کرده و خروجی همواره به صورت ترکیبی از روی ورودی‌ها حاصل شده و مدار حافظه ندارد.

می‌توانیم سیگنال‌های ورودی و خروجی این شبیه‌سازی را در نرم‌افزار GTKWave نیز بررسی کنیم:



شکل ۳: شکل موج سیگنال‌ها در محیط GTKWave برای کد اصلی

همان‌طور که مشاهده می‌شود، خروجی مالتی‌پلکسر به تغییر sel واکنشی نمی‌دهد. این موضوع باعث شده که عملکرد آن اشتباه باشد و عملاً حافظه داشته باشد، در صورتی که یک مدار ترکیبی است. شکل موج‌ها برای نسخه‌ی اصلاح شده نیز به این صورت می‌باشد:



شکل ۴: شکل موج سیگنال‌ها در محیط GTKWave برای کد اصلاح شده

که روشن است در این نسخه، با عوض شدن مقدار sel، خروجی نیز تغییر می‌کند و عملکرد مدار صحیح است.